

Stress Testing and Scaling WordPress on a \$12 VPS

From server crash to enterprise scale — a live-fire DevOps exercise

Nedko Hristov · Senior DevOps Engineer @ Nemetschek Bulgaria
WordPress Europe 2026 · Kraków



github.com/NedkoHristov/WordPress-Europe-2026

Who am I?

- **Nedko Hristov** — Senior DevOps Engineer @ Nemetschek Bulgaria
- Building and breaking WordPress infrastructure since 2012
- Contributor to the Bulgarian WordPress community
- Speaker at dev.bg, WordCamp Sofia, and now — WordPress Europe 🎉

Today's promise:

By the end of this talk, you'll know exactly which knob to turn when your WordPress site falls over — and why it works.

What we're working with

The "VPS"

- 2 vCPU · 2 GB RAM · 20 GB SSD
- ~\$12/month (Hetzner CX22 equivalent)
- Docker Desktop (local, same constraints)

The WordPress site

- 2,500 posts + 500 products
- WooCommerce active
- ~6,000 revisions in the database
- "Real" bloat — expired transients, orphaned meta

The load generator

- k6 · open source · Grafana Labs
- Ramping VU scenario: 10 → 500 VUs
- Named: **Black Friday ramp**
- All metrics live in Grafana

The repo

```
git clone github.com/NedkoHristov/WordPress-Europe-2026
docker compose up # everything starts
```

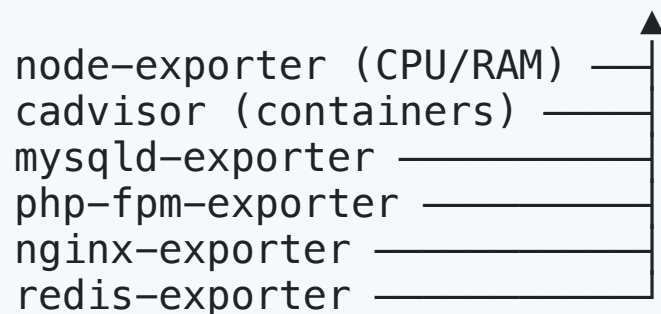
The 4-level scaling journey

Level	Stack	Key change
0	Apache + mod_php	Baseline — we crash it here
1	Nginx + PHP-FPM + OPcache + Redis	Event model + bytecode cache
2	Level 1 + FastCGI page cache + MariaDB tuning	95% of requests skip PHP
3	Level 2 + Cloudflare CDN	Edge absorbs the load
4	Level 2 + Simply Static export	WordPress as a CMS, not a runtime

Same hardware. Same site. Same load. Wildly different results.

Observability stack

k6 —remote-write—▶ Prometheus —▶ Grafana dashboards (live)



6 dashboards auto-provisioned:

Overview · PHP-FPM · MariaDB · Nginx Cache · Redis · k6 Live

Everything you see on screen is real data from the running containers.

ACT I

The Crash

Level 0 — Apache + mod_php

Level 0 architecture



What this means:

- Each Apache worker = 1 full PHP process in RAM
- 2 GB RAM → ~40–50 usable workers
- Worker 51 = **wait in queue**
- Queue full = **503 Service Unavailable**

DEMO — Black Friday ramp begins

```
make load-crash    # 0→500 VUs over 14 minutes
```

Watch: <http://localhost:3000/d/wp-overview>

Minute	VUs	What's happening
1	10	All green  p95 < 200ms
3	50	CPU climbing, load > 2 
4	80–100	FPM saturated, queue forming 
4:30	100	MySQL threads spike 
5	100–150	CRASH — 503s, load > 10 

The money shot

CPU: 100% · Load: >10 · RAM: near zero · Error rate: >50%

(screenshot: act1-crash-overview.png)

What killed it?

1. **Apache prefork** — every idle connection holds a PHP process
2. **mod_php** — no opcode cache, compiles every request
3. **No Redis** — every `get_option()` hits the database
4. **Untuned MariaDB** — 128MB buffer pool on a bloated DB

The Apache prefork math

$$\begin{aligned}\text{Max safe workers} &= \frac{\text{RAM} - \text{OS overhead}}{\text{RAM per PHP process}} \\ &= \frac{2048 \text{ MB} - 512 \text{ MB}}{32 \text{ MB}} = \mathbf{47 \text{ workers}}\end{aligned}$$

Worker 48 goes into the queue.

Worker 100 gets a 503.

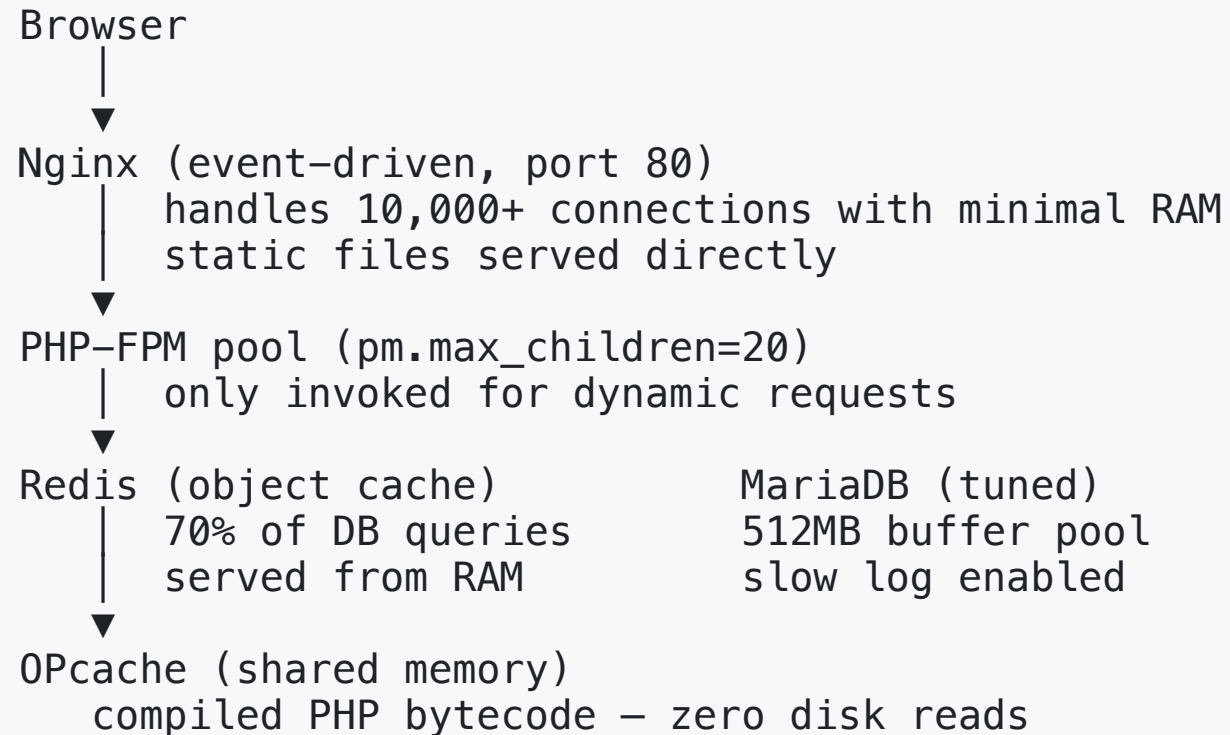
The real problem: Apache keeps idle workers alive — fully loaded PHP processes doing nothing, consuming 32MB each.

ACT II

Stop the Bleeding

Level 1 — Nginx + PHP-FPM + OPcache + Redis

Level 1 architecture



OPcache: the biggest single win

Without OPcache (Level 0):

```
Request → PHP reads file from disk → lexer → parser → compiler → execute  
      ↑ repeated EVERY. SINGLE. REQUEST.
```

With OPcache (Level 1):

```
Request → PHP reads bytecode from shared memory → execute  
      ↑ compiled once, cached forever (until deploy)
```

Result: 98%+ cache hit rate within 30 seconds

`opcache.validate_timestamps=0` — never re-check disk in production

Redis object cache

Without Redis:

```
Every wp_query → MySQL → disk I/O → parse → PHP array  
wp_get_option() called 300+ times per request
```

With Redis:

```
wp_get_option() → Redis GET → RAM → done (microseconds)  
~70% of DB queries served from memory
```

FPM pool math:

$$\text{Concurrent PHP workers} = \frac{\text{RAM for PHP}}{32 \text{ MB}} = \frac{512 \text{ MB}}{32 \text{ MB}} = 20 \text{ workers}$$

20 workers handle 200 VUs because Nginx queues connections.

Workers do real work — they don't idle holding connections.

DEMO — Same load, Level 1

```
make level-1    # stops Apache, starts Nginx + FPM + Redis
make load-crash
```

Metric	Level 0	Level 1
Max VUs before crash	~80	350+
p95 latency at 100 VU	timeout	~180ms
PHP-FPM queue depth	N/A (crash)	0
OPcache hit rate	0%	98%+
Redis hit rate	0%	~70%
CPU at 200 VU	100% (dead)	~60%

(screenshot: act2-level1-fpm-workers.png)

ACT III

Cache Everything

Level 2 — FastCGI page cache + MariaDB tuning

FastCGI page cache

The insight: most WordPress pages are **identical** for every anonymous visitor.

```
fastcgi_cache_path /var/cache/nginx
    keys_zone=WORDPRESS:100m
    inactive=60m;

fastcgi_cache_key "$scheme$request_method$host$request_uri";
```

Bypass rules (the important part):

```
# Never cache: logged-in users, cart, checkout, POST requests
if ($http_cookie ~* "wordpress_logged_in|woocommerce_cart") {
    set $skip_cache 1;
}
```

Result: X-Cache-Status: HIT — PHP never runs

Watching the cache warm up

<http://localhost:3000/d/wp-nginx-cache>

Time	HIT %	What's happening
0:00	0%	Cache cold — all MISS
0:30	~40%	k6 starting to re-request URLs
1:00	~70%	Frequently visited pages cached
2:00	90%+	Cache hot ← KEY SCREENSHOT

PHP-FPM Offload panel:

- Total RPS line: high (all traffic)
- PHP-FPM RPS line: near zero (only MISSes)

The gap between the two lines = **your cache doing its job**

MariaDB tuning

Level 0 (baseline):

```
innodb_buffer_pool_size = 128M    # too small for our bloated DB
```

Level 2 (tuned):

```
innodb_buffer_pool_size = 512M      # fits most of the DB in RAM  
innodb_flush_log_at_trx_commit = 2  # fsync once/sec vs every commit  
slow_query_log = 1  
long_query_time = 0.5                # catch queries > 500ms
```

Result on Grafana MariaDB dashboard:

- Buffer Pool Hit %: **> 99%** (reads from RAM, not disk)
- Threads Running: **< 5** (was spiking to 50+ at crash)

Level 2 results

500 VU · CPU: 15% · Load: 0.8 · Error rate: 0% · TTFB: 8ms

(screenshot: act3-cache-hot.png)

"95% of requests never touch PHP.

The remaining 5% are admin, cart, logged-in users.

TTFB went from 800ms to 8ms.

The \$12 VPS is practically bored."

ACT IV

The Hybrid Static Leap

Level 4 — WordPress as a CMS, not a runtime

The static export concept

```
make static-build    # wget crawls every page → static/export/  
make level-4         # Nginx serves static files from export/
```

What happens:

```
Build time:    WordPress renders every page once → saves HTML to disk  
Runtime:      Nginx reads HTML from disk → zero PHP, zero MySQL  
              Dynamic parts (cart, checkout) → proxied to WP origin
```

The "dynamic islands" pattern:

```
Static Nginx (port 8090)  
├── /                → static HTML (0ms PHP)  
├── /shop/           → static HTML (0ms PHP)  
└── /wp-json/        → proxy to WordPress origin (dynamic)  
    /wc/store/cart   → proxy to WordPress origin (dynamic)
```

DEMO — Side by side

Open two tabs:

- <http://localhost:8080> — WordPress origin (Level 2)
- <http://localhost:8090> — Static export

```
# Hammer only the static site
k6 run --vus 500 --duration 2m browse.js
  --env BASE_URL=http://static-site:80
```

Metric	Level 2	Level 4 (static)
p95 latency	45ms	< 5ms
CPU at 500 VU	15%	< 2%
PHP processes running	~8	0
Cart works?	✓	✓ (dynamic island)

The mic drop

500 VU · p95: 4ms · Error rate: 0% · WordPress CPU: 0%

(screenshot: act4-k6-static.png)

"WordPress as a CMS, not a runtime.
Write your content in WordPress.
Serve it from Nginx."

The full picture

Level	Stack	Max stable VUs	p95 latency	Cost change
0	Apache + mod_php	~50	crashes	baseline
1	Nginx + FPM + OPcache + Redis	300+	~180ms	\$0
2	Level 1 + FastCGI cache + MariaDB	500+	~45ms	\$0
3	Level 2 + Cloudflare CDN	∞ (edge)	~20ms	~\$0–20/mo
4	Static export	500+	~4ms	\$0

Every improvement cost \$0 in hardware.

It was configuration, architecture, and understanding bottlenecks.

Lessons learned

1. Measure first, optimize second

"You can't fix what you can't see."

Install Grafana + Prometheus before you need it.

2. The three layers of WordPress caching

OPcache	→ PHP bytecode in shared memory	(always on)
Redis	→ DB query results in RAM	(WP object cache)
FastCGI	→ Full HTML pages on disk	(anonymous visitors)

3. pm.max_children is not "more = better"

$$\text{pm.max_children} = \frac{\text{Available RAM for PHP}}{\text{RAM per worker}}$$

Over-provision = OOM killer. Under-provision = queue. Do the math.

Lessons learned (cont.)

4. The database is usually the real bottleneck

- Check `innodb_buffer_pool_size` — should be 70% of available RAM
- Enable `slow_query_log` — find queries > 500ms
- Watch **Threads Running** — if it spikes, you're in trouble
- Run `wp db optimize` regularly — orphaned meta and revisions add up

5. Static ≠ broken dynamic

The "dynamic island" pattern: serve 95% of pages statically, proxy 5% (cart, checkout, account) to WordPress.

6. Cloudflare is free for personal sites

Zero Trust tunnel + cache rules = your laptop serves the world.

The tools (all free, all open source)

Tool	What it does
k6	Load generator — scriptable, Prometheus output
Prometheus	Time-series metrics storage
Grafana	Dashboards — 6 pre-built in the repo
Redis	Object cache — <code>wp redis enable</code>
PHP OPcache	Bytecode cache — already in your PHP
Nginx FastCGI	Full-page cache — built into Nginx
Simply Static	WP plugin — static site export
Cloudflare	CDN + Zero Trust tunnel — free tier

Repo: `github.com/NedkoHristov/WordPress-Europe-2026`

`docker compose up` — everything starts. Every demo is reproducible.

Thank you!

Nedko Hristov

Senior DevOps Engineer @ Nemetschek Bulgaria

 github.com/NedkoHristov/WordPress-Europe-2026

 [@nedkohristov](https://twitter.com/nedkohristov)

 linkedin.com/in/nedkohristov

Questions?

*"The best time to add monitoring was before the crash.
The second best time is now."*